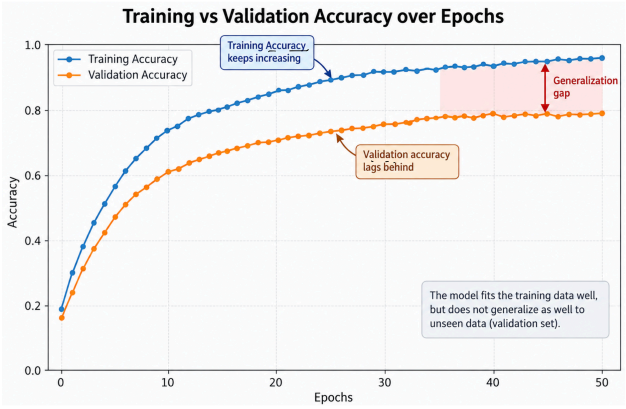
Hyperparameter Tuning: Finding Better NN

Saturday, 10 October 2026 8:44 PM

Optimizers :

1) SGD , Mini Batch GD , Vanilla GD
2) GD with Momentum
3) RMSProp.
4) Adam (Momentum + RMSProp).

Q. How to build better NN ?



} Overfitting

→ Regularisation : L_1 , L_2

→ Early Stopping

→ Simpler Model.

* Categorical Columns.

Encoding :

Gender. → Male : 0
→ Female : 1

Income level → High : 2
→ Medium : 1
→ Low : 0

} Ordinal

City → A
→ B
:
→ Z

} 1000 values. → 1000 additional columns with OHE

Probability

$$TE(c) = \frac{1}{n_c} \sum y_i$$

City	y (target)
A	1
A	0
A	1
A	0
B	1
B	1
B	0

⟹

(TE) City	y
0.5	1
0.5	0
0.5	1
0.5	0
0.66	1
0.66	1
0.66	0

$A = \frac{2(y=1 \mid city=A)}{4(city=A)} = 0.5$

$B = \frac{2(y=1)}{3(city=B)} = .66$

* Standardisation -

Distance in km ⟶ 0 – 30 km.

Weight in gm ⟶ 0 – 10,000

Standardisation puts the features on comparable scales.

$$z = \frac{x - \mu}{\sigma}$$

(x = data point; $\frac{x - \text{mean}}{\text{std.}}$)

↓

mean → 0

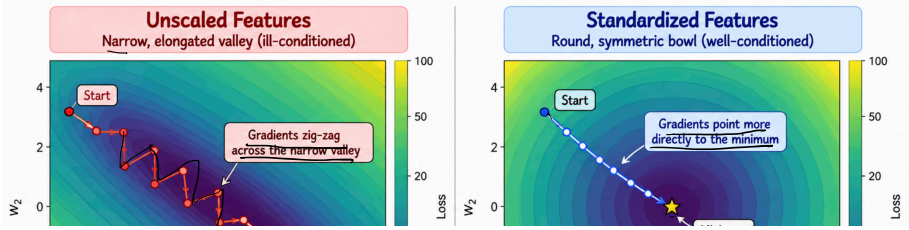
S.D → 1.

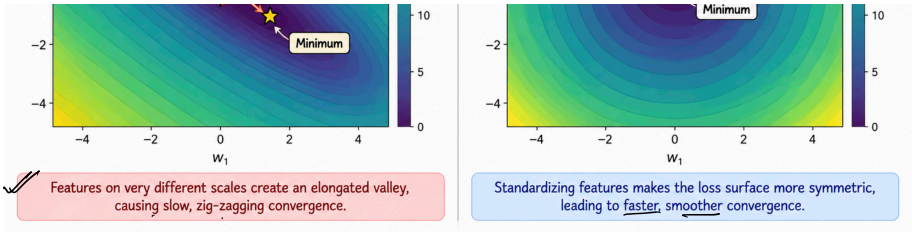
→ Standardisation is robust to outliers

→ Standardised features has mean of 0 and s.d of 1.

Quiz 1 $\frac{6}{40} = .15$

$$\text{Normalisation} = \frac{x - x_{min}}{x_{max} - x_{min}}$$ → impacted by outliers.





Model will converge faster. Faster training.

Class → 0/1 , Loss → Binary Cross Entropy.

Mean Squared Error.

Probabilities → 0.9 , 0.7 , 0.6.

Q. Why should we not use MSE in case of binary classification?

⇒ Examples with mathematical calculation. (offline Read).

* L_1 vs L_2 Regularisation.

$L_1 \rightarrow \|W\|$ → Features with less importance get 0 weights. (Feature selection)

$L_2 \rightarrow W^2$ → Features with less importance get smaller values, but not 0.

* L_2 Regularisation.

A weight with very high magnitude can make the model highly sensitive to a feature.

L_2 adds penalty for higher/ large weights

$$L_{total} = \underbrace{\overbrace{L_{data}}^{CCE}}_{\text{Loss over the data points.}} + \frac{\lambda}{2m} \sum (W^l)^2.$$

L_{data} : original prediction loss

W^l : weights in layer l

m : # of data points

λ : regularisation strength (Hyperparameter).

$$\frac{\partial L_{total}}{\partial W} \rightarrow dW_{total} \text{ (Chain Rule).}$$

for each epoch:

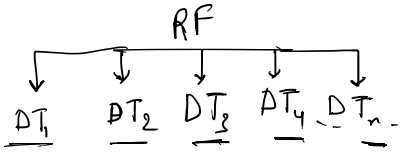
$$W_{new} = W_{old} - \eta \left(dW_{total} + \frac{\lambda}{m} W \right).$$

$\lambda = 0$ → No regularisation. (No L2).

λ = Moderate → Large weights are discouraged

Very large λ (↑ hyperparameter) → Weights will be pushed towards 0. model might underfit.

$D_{m \times n}$



Bootstrapping + Aggregation.

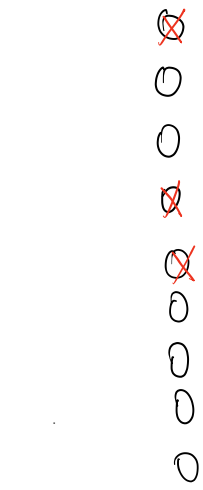
↑

Random Sample of $D'_{m' < m}$ and $n' < n$.

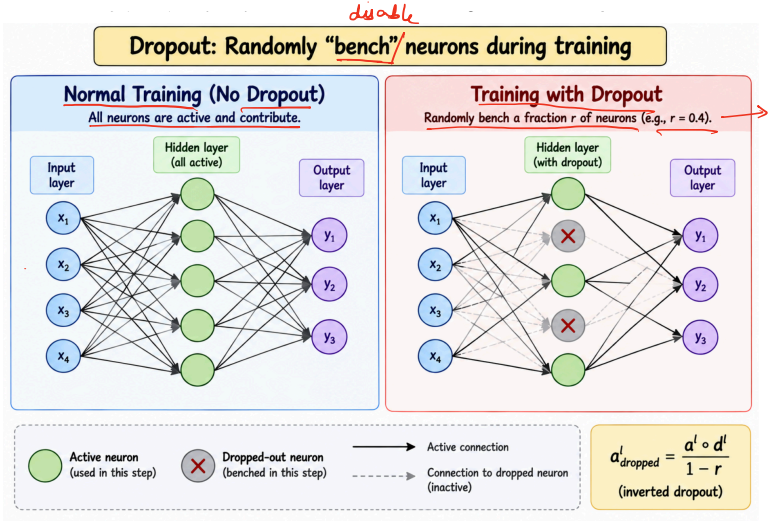
* Dropouts

After every layer, you can apply dropout.

dr = 0.3 → 30% of neurons in the layer are disabled.



disable

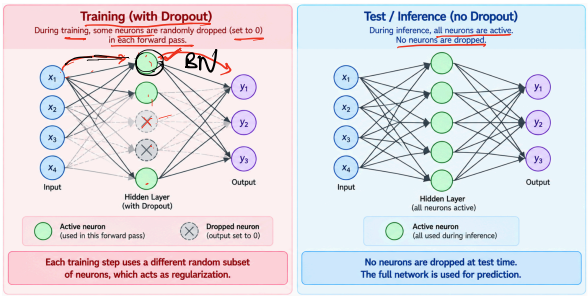


1000 neurons.

$r = 0.25$

how many neurons are disabled ?

$0.25 \times 1000 = 250$ neurons disabled.



total neurons = 256

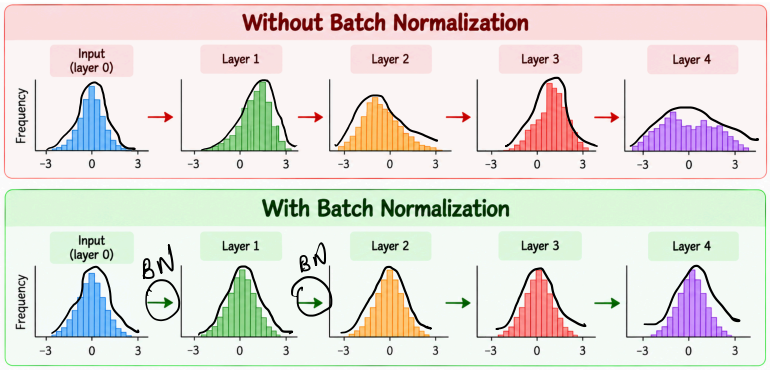
$r = 0.25$

Neurons dropped = $256 \times 0.25 = 64$

* Batch Normalization.

... can

In a deep network, small changes in the output from 1 layer get magnified as we propagate it through multiple layers.



Normalize the output of each layer before passing it to the next layer.

Steps in BN : (pre activation)

1) Calculate batch mean

$$\mu_B = \frac{1}{n} \sum_{i=0}^{n} z_i$$

2) Calculate variance

$$\sigma_B^2 = \frac{1}{n} \sum_{i=1}^{n} (z_i - \mu_B)^2.$$

3) Normalize

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

↑ very small value to ignore division by 0.

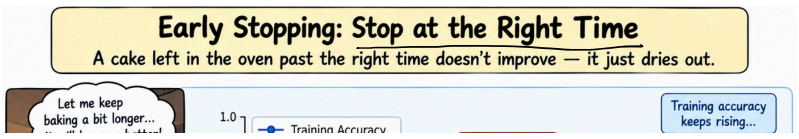
4) Scale and shift

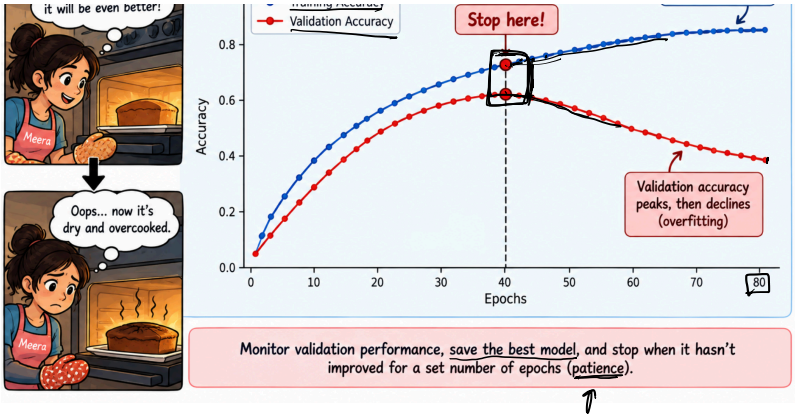
$$y_i = \gamma \hat{z}_i + \beta$$

y_i: activation, γ: scaling, β: shifting.

Batch Norm is applied after every layer

* Early Stopping.





Monitor validation performance, save the best model, and stop when it hasn't improved for a set number of epochs (patience).

↑

if it does not improve for n' iterations, stop training

↑

patience